Data Encapsulation & Transmission in Computer Networking

by Joseph (“Joey”) Lincoln
ENG249 – Technical and Professional Writing I, Professor Angela Haas

A technical report on the process of information abstraction and communication in modern computer networks, with explorations in how this process unfolds within both the OSI and TCP/IP networking models, in addition to a comparison of the networking models themselves. Forewarning – only because of the precision and specificity in the scope of this report, it is recommended that the reader have *at least some* background in using a computer and basic, online technologies such as e-mail or a web browser. Some limited understanding of simple computer networking terminology would also be advantageous in achieving competency of the subject matter more quickly. Terms like IP address, router, LAN vs. WAN, bandwidth, etc. would be useful to at least be aware of beforehand. That being said, this report will be written with the layperson audience in mind – the safe assumption being that most laypeople have a fair amount of experience using an Internet connected device of some sort, whether it’s a desktop computer, a laptop, a tablet, or a smartphone.

A History and Overview of Computer Networking

In the same decade that the revolutionary *von Neumann and Harvard computing architectures were introduced to the world, the first modem was developed at the Air Force Cambridge Research Center (AFCRC) around Boston, Massachusetts in 1949. The name for the **modem** technology is, interestingly enough, a portmanteau of the two main processes which it performs: **MOD**ulation of data into analog signals, and **DEM**odulation of analog signals into digital information (the reverse operation). Its original purpose was in the domain of purely military technology, as the intended use case was only to transmit radar signals. However, this new technology had another, unforeseen application – the implementation of this modulation and demodulation process with computers and telephone lines allowed sound signals transmitted via the landlines to be used as a medium for digital, computerized data transmission as well. Within ten years, Bell Telephone / AT&T would acquire the rights to commercialize the telephone line infrastructure towards this end. Within those same ten years, the development and deployment of the Semi-Automatic Ground Environment (or SAGE air defense system) will be spearheaded by the nation’s top academics, the International Business Machines (IBM) corporation, and the United States Air Force. The Massachusetts Institute of Technology orders construction of a new building, Lincoln Laboratory, where most of the early research and development for the SAGE air defense system and the accompanying IBM AN/FSQ-7 Combat Direction Central computer (or simply, Q7) occurs. Though impressive for the time, this system would look primitive by today’s metrics; a rudimentary network of 20 something IBM Q7 machines across North America, connected by telephone wires adapted to computers through modems, all manned by operators whose job is to stay on the alert for nuclear, Soviet bomber planes. For the 1950’s, this was yet another massive leap forward, not just for computing, but for what would become computer networking as well.

*von Neumann and Harvard computing architectures: the differing designs created in the 1940’s to allow for computer systems to be able to both store and execute sets of data and instructions internally; this was in opposition to earlier computers like the vacuum tube powered ENIAC which lacked the implementation of a stored-program or data concept and relied on external switchboard operators to “program” it by manually plugging and un-plugging wires.

The invention of the intercontinental ballistic missile would soon render nuclear bomber planes, and therefore the SAGE air defense system, obsolete. Before the SAGE system is retired within about the next decade, IBM and American Airlines collaborate on the Semi-Automated Business Research Environment (or SABRE system), which is adapted from the SAGE air defense system for commercial use. In this case, for reserving a seat or ticket on a flight. Various businesses, financial institutions, universities, and military installations set up “time-sharing communities”, the first publicly available, multi-user, connected computer networks.

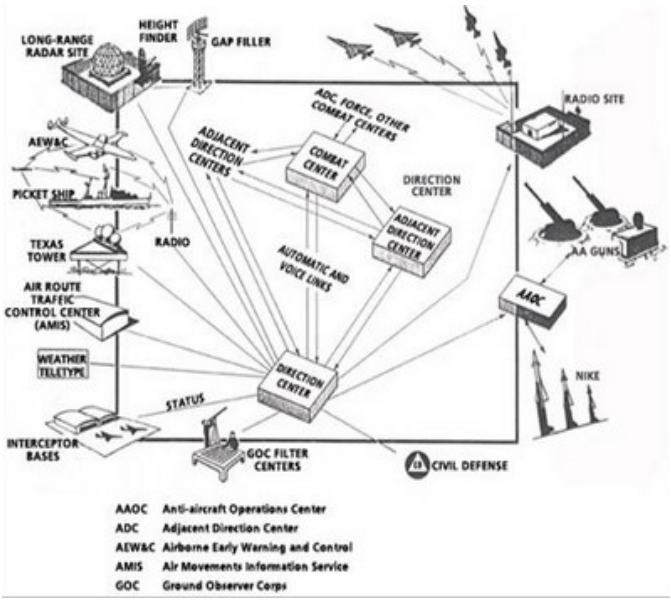


Figure 1 – a graphical representation of the SAGE air defense system's high level design schema

In 1969, [Δ]ARPANET goes live, which is a truly major milestone for modern computer networking. It was the very first *packet switching* network, and the foundations, development, and infrastructure it laid ended up paving the way for what is now the Internet as we know it. ARPANET originally used specialized nodes connected to user machines, known as Interface Message Processors (IMPs), to allow for a kind of proto-TCP/IP networking communication to occur between computers linked by IMPs. These unique processor nodes were, effectively, the technological precursors to the networking devices we use now known as routers. By the 1980’s, IMPs would be phased out of ARPANET completely, in favor of the more modern TCP/IP networking infrastructure which most computing experts are familiar with today.

Networking Models (TCP/IP vs. OSI)

This is where the basis and foundation for modern computer networking really begins. The **TCP/IP** networking framework, also known as the **Internet Protocol Suite**, is the most prevalent model in use for data communication on modern networks like the Internet, with another prevalent one being the **Open Systems Interconnection model (OSI)**. A key difference between the two is how the process of data communication is structured.
The OSI model is a theoretical, abstract framework, comprising seven layers of protocol-independent networking phases which govern how, when, and where data transmission and encapsulation occur. Each layer represents a specific function in the data transmission process. In contrast, TCP/IP is a practical, concrete model with only four layers – this is the model used in actual networking practice. It simplifies OSI’s seven layers, combining several of them while still supporting the overall data communication process.

[Δ]ARPANET: Advanced Research Projects Agency Network; development primarily overseen by the United States Department of Defense’s Advanced Research Projects Agency, with key collaborators.

TCP/IP's highest level, **application** combines OSI's application, presentation, and session layers. The next two levels, transport and network are the same across both models. TCP/IP's lowest layer – network interface – combines OSI's data link and physical layers. The processes that occur at each layer will be explained in detail step-by-step in this process description. To understand data encapsulation, it can be useful to follow an example through the network layers. For the purposes of this technical report / process description, the working example of an e-mail being sent will be used in order to help give the reader a more direct understanding of how the process occurs, and what functions each layer entails.

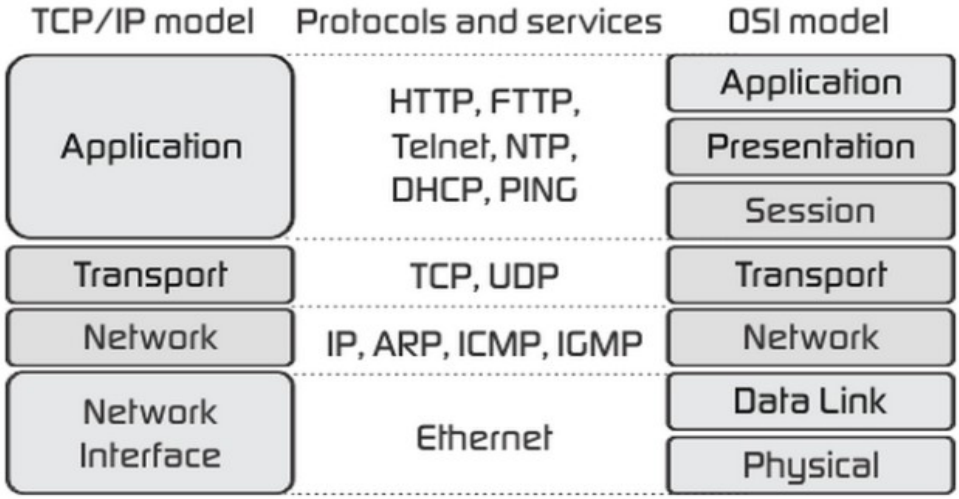


Figure 2 – a direct, graphical comparison between the TCP/IP model and the OSI model

Application Layer

The application layer is where software interacts with the network. As mentioned, this single layer of the TCP/IP model corresponds to the three upper layers of the OSI model. These layers prepare user-created data for transmission across a network. In the case of our working example, the data generated by a user is an email.

At the highest level of the **Application Layer** (which corresponds to **Layer 7 of the OSI model** in which it is also called the Application Layer), communication with a network begins when a user composes an email using an email application (think Outlook, Gmail, Mozilla, etc.). The application interacts with a mail server using a communication protocol such as **Simple Mail Transfer Protocol (SMTP)**. Communication protocols are standardized rule sets that dictate how data is exchanged on a network. Perhaps the most common protocol is **Hypertext Transfer Protocol, or "HTTP"**. With HTTP, a user sends a request to a webserver through a browser application like Chrome, and a web page is sent back in return. There are many such communication protocols with a multitude of functionalities. HTTPS, for example, is exactly the same as HTTP with the added S for "security", meaning that data is not only transmitted, but also *encrypted.*

Returning to the email example: at this point the email message, composed of the sender's address, the recipient's address, subject line, message body, and attachments, has been sent to the mail server via SMTP. For the purposes of this report, we will imagine that the final, desired destination of the email is in fact a Google SMTP / [Σ]POP3 mail server located at IP address 74.125.24.108.

The next step occurs at the OSI's **Presentation Layer (Layer 6)**, which is responsible for ensuring that the data is formatted in a way that the receiving system can understand. This layer may translate data formats, compress the data, or encrypt it before transmission. For email communication, attachments are often encoded using standards such as **Multipurpose Internet Mail Extensions (MIME)**, which converts binary files into text-based representations that can be safely transmitted through email systems. If secure communication is used, encryption may also be applied through protocols such as Transport Layer Security. By the end of this stage, the message is fully formatted and prepared for secure and consistent interpretation by the receiving device.

The **Session Layer (Layer 5)** then manages the communication session between the sender's system and the mail server. This layer establishes, maintains, and eventually terminates the connection used to transmit the email. During this process, the email client initiates a session with the mail server and exchanges commands defined by SMTP**.** These commands coordinate the transfer of the message, specifying the sender, the recipient, and the point at which the email content is transmitted. Once the mail server receives and acknowledges the message, the session is closed. After these three layers complete their tasks, the email exists as fully prepared Application Layer data. It is then passed to the **Transport Layer**, where protocols such as Transmission Control Protocol begin the next stage of encapsulation.

Transport Layer

The main goal of the Transport Layer is to ensure reliable and efficient end-to-end communication between devices by delivering data accurately and in the correct order. The Transport Layer manages communication between devices and ensures that data is delivered correctly. It establishes a connection between devices, maintains it during communication, and properly terminates it when finished. It also breaks data into smaller segments for transmission and reassembles them at the destination. It also provides error control by detecting and retransmitting lost or corrupted data. On top of all of that, it controls the flow of data to prevent overwhelming the receiver and uses **port numbers** to direct data to the correct application. The Transport Layer uses segments which are units of data that contain a portion of the original data along with header information. This layer is all about protocols and how they work. **Protocols** are sets of rules and standards that define how data is transmitted and received, so devices can communicate correctly over a network. Within this layer there are a couple of protocols, but the predominant ones are TCP and UDP.

[Σ]POP3: Post Office Protocol, version 3

The Transmission Control Protocol (TCP) is a protocol that provides reliable communication between devices over a network. This protocol is known to be reliable and connection-oriented, meaning it does not lose segments as much as other protocols. To be connection-oriented, it means that every time it attempts to connect with another device it starts using a handshake before any data is sent so both sides are ready. TCP specifically uses the **three-way handshake** to complete this task. Within the three-way hand shake we have **Synchronize (SYN) and Acknowledgement (ACK)**. SYN segment is sent by the client, and the host sends a SYN + ACK. Next, the client sends another ACK, and after this we have completed our 3-way handshake (Kozierok). In Figure 3 each arrow represents a segment that was sent.

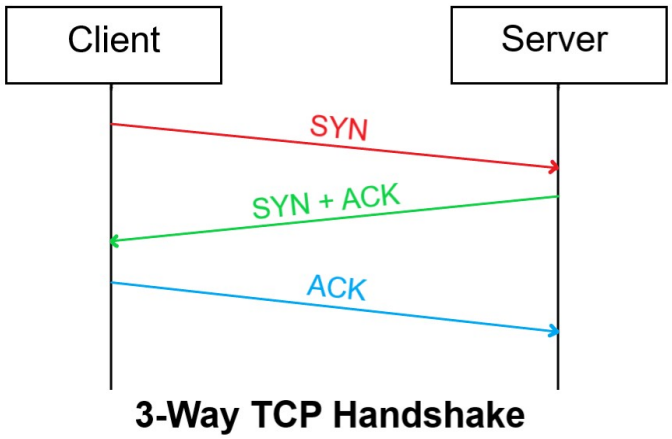


Figure 3 – A visual of the TCP/IP 3-way Handshake

The TCP header plays an essential role within its protocol because it contains important information that helps manage and control the delivery of the data. It contains source ports and destination ports. These are numbers that identify the sending application and the receiving application, so data is delivered ensuring that the data is routed. To keep track of the order of the data, TCP uses a sequence number. The Acknowledgement number shows how much data has been sent successfully. The checksum is another parameter of the TCP header that checks if the data was corrupted. The urgent pointer shows where the urgent data lies within the segment. If any data or segments were lost, TCP can identify this issue with all the information above. For example: If the sender does not receive an ACK at a certain time, it assumes the segment was lost and will resend it (Kozierok). Keep in mind that not all of these fields within the TCP header have to be used. Most of the time these headers do not have to be full length.

In the context of our email example, heading to some Google SMTP / POP3 at an IPv4 address of 74.125.24.108, TCP is the protocol of choice. This is because we must ensure that the transmission and delivery of our email is stable, with each packet arriving to its destination, reliably, and in sequence. Protocols within the Transport Layer are not aware of the kind of data that they are carrying though; whether or not its an email is irrelevant. The entire TCP handshake process occurs with the data encapsulated into unreadable bits. The Transport Layer does exactly as its name suggests – it segments the bits of raw data into portions, verifies the connection, and *transports* the data.

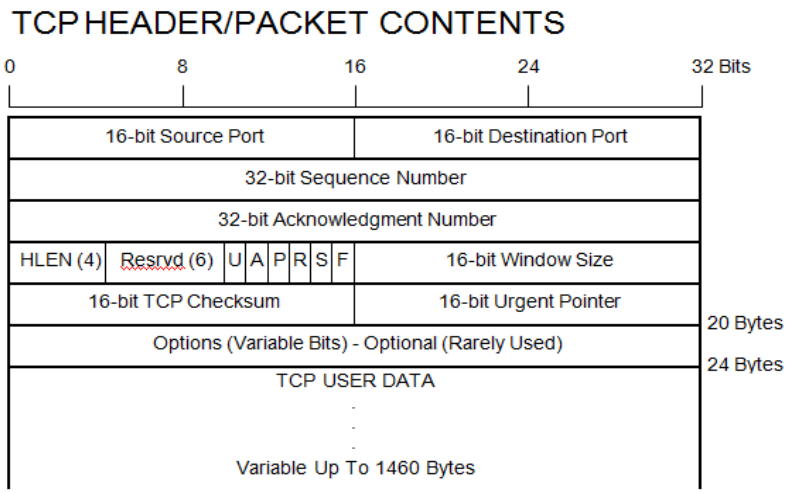


Figure 4 – TCP/IP Header

User Datagram Protocol (UDP) is a simple, connectionless transport layer protocol that sends data quickly without establishing a connection first (Kozierok). This protocol is really meant for fast low latency communication where speed is the main priority and is used for certain streaming services, video chats, and online gaming. This speed is accomplished without establishing a connection and containing less data in the UDP header compared to the TCP header. The UDP header only contains the source port, destination port, length, and checksum.

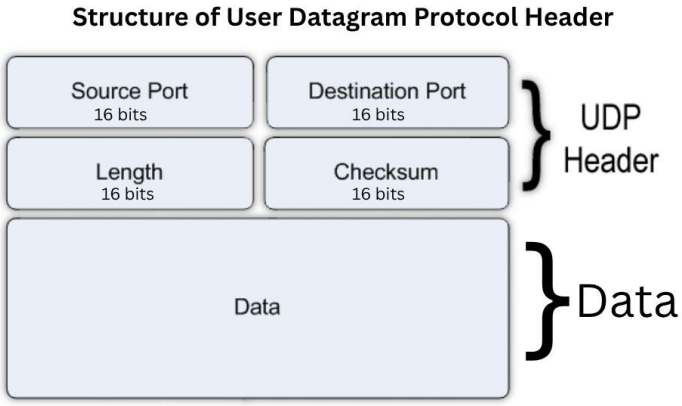


Figure 5 – UDP Header

It does not guarantee delivery, order, or error recovery, meaning segments can be lost, duplicated, or arrive out of order, but this makes it much faster and can be more efficient than TCP. Although the UDP header does not support error recovery or order, it provides the checksum for applications to help mitigate this issue of UDP. Even with this system, UDP is still not as reliable as TCP, but it definitely makes a difference.

Overall, the Transport Layer is a small yet extremely crucial part of the data encapsulation process as it takes data from the application layer and breaks it into segments ensuring delivery to the correct application on the destination device using port numbers. Although without the network layer, the transport layer would not be able to function.

Network Layer

The **Network Layer**, in both the TCP/IP and OSI models, is responsible for the logical addressing and delivery of data to its destination. The most common network layer protocol, and the only one relevant to this process description is the **Internet Protocol**, which uses the Internet Protocol (or IP) addressing scheme. IP addresses indicate the "location" of a specific device on a network. Every device that connects to the internet is assigned an IP address.

This layer then converts the segments into units called "**packets**" by attaching a header containing the source and destination IP addresses to each one. In our example, the destination IP address of 74.125.24.108 would be included in this header. The source IP address corresponds to the sender's device. The network layer uses both of these addresses to determine the ideal path for the data to take.

On the internet, the optimal path from one IP address to another for data transfer is not always simple. Packets may be sent to different nodes (or devices) on their way to their destination. Furthermore, the packets that comprise a single email/piece of data may be sent by different paths and arrive at the destination out of order. This is why the sequence number is attached at the transport layer, allowing the receiving system to properly reassemble the data.

The structure of the internet itself can be described as a **mesh network**, meaning that devices (particularly routers) are interconnected through multiple possible pathways rather than a single fixed route. This design increases reliability and efficiency, as data can be rerouted if a particular path becomes congested or unavailable. When a packet is sent, routers use **routing tables** and algorithms to determine the best available path based on factors such as distance, network congestion, and link availability. As conditions on the network change, these paths can be dynamically adjusted in real time.

Unlike the Transport Layer, the Network Layer does not guarantee delivery or handle retransmission of lost packets. The Internet Protocol is considered a **best-effort delivery** protocol, meaning it attempts to deliver packets but does not provide error recovery. Instead, basic error detection is performed using the header checksum. If an error is detected, the packet is typically discarded. Ensuring that all data is correctly received and retransmitted if necessary is the responsibility of higher layers, particularly the Transmission Control Protocol at the Transport Layer.

By the end of this stage, each segment has been encapsulated into packets with the appropriate logical addressing and routing information. These packets are then passed down to the **Data Link Layer**, where physical addressing and access to the transmission medium are handled for the next phase of transmission.

Network Interface Layer

The final and lowest level of the TCP/IP model is the **Network Interface Layer**, which combines the last two layers of the OSI model (**Data Link and Physical**). As the name might infer, this layer relates to how data is handed off from the network layer to the physical devices and media which ultimately act as the endpoints to which data will be received / delivered. In contrast to the (comparably) tangible nature of this layer, it will actually be beneficial to explain this layer starting from somewhere a bit more abstract. Before data can reach the Physical Layer of the OSI model, it must pass through the Data Link Layer where a couple of crucial operations must take place to ensure safe and proper delivery of the packets.

While the Network Layer handled packet forwarding across interconnected, wide area networks (also known as WANs), the Data Link Layer is not concerned with such frivolities. Here, the entire point is to package the true, final destination (74.125.24.108, the Google SMTP / POP3 mail server) within a sort of "Russian nesting doll" sequence, if you will, that will only worry about transmission of the data across local area networks (or LANs). The destination IP address is not relevant to the delivery of the data across various LANs, over a single or multitude of WANs, until the packets have been delivered right to the doorstep of the destination address (some Google LAN server). This is a highly nuanced, and often misunderstood, aspect of networking communications at this level which warrants the need for clarity here before diving in any further.

Packets coming in from the Network Layer (Layer 3) must undergo a translation process of sorts, coined **framing**, before they can proceed any further. During this process, the raw bits of the data packets being sent in are wrapped in a "frame" which creates a readable, manageable, and most importantly – controllable version of the packets and data. This makes it much easier for the LAN hardware and devices to work with everything being sent in. Besides a translation, the frames also enforce a strict size constraint – Ethernet standards have a minimum of 64 bytes per frame and a maximum payload of 1500 bytes. But in addition to just the raw data being sent in, additional, Ωmetadata must be added to the frames to give context and information about the packets. This includes a 14-byte Ethernet header, which has data regarding the local network's broadcasting device, and a 4-byte Frame Check Sequence (FCS) as the tail of every Ethernet frame. The FCS is an integrity check, to alert the receiving device as to whether or not any bits were corrupted or lost en route to the destination.

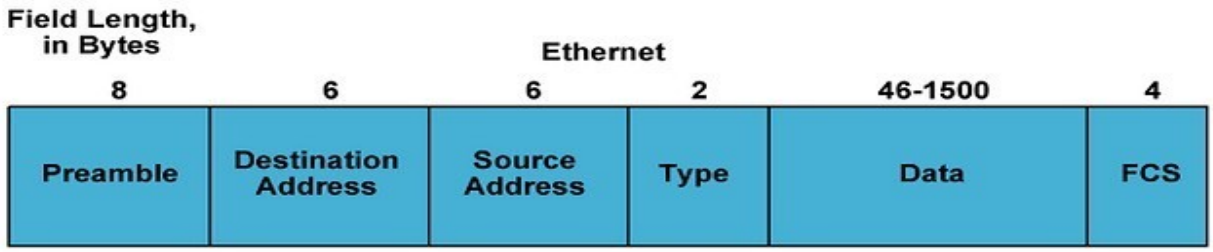


Figure 6 – Byte sizes and partitioning of a typical Ethernet frame shown

Ωmetadata: information about the data itself (method of delivery, type of data, size, etc.)

Another one of the most vital protocols at this layer has been dubbed the **Address Resolution Protocol, or ARP**, which effectively operates at the Data Link Layer (Layer 2) of the OSI model, though it is basically a bridge or connector between this layer and the layer above it, the Network Layer. The ARP has a very, *very* important job, and that is to assign dynamic IP addresses to their corresponding **Medium / Media Access Control, or MAC, addresses**. All Internet capable devices have a MAC address, which uniquely identifies that device as that device – it is encoded as part of every device's Network Interface Controller, or NIC, during manufacturing. When a router or switch or other networking device receives the desired destination IP address, in our case 74.125.24.108, it sends out a message to all connected devices on the LAN basically asking, "Hey! Which one of you has a dynamic IP address of 74.125.24.108?!". Whichever device recognizes that as their outgoing, external IPv4 address responds with their identifying MAC address, and the final handoff can occur. If the destination address has not been reached however, and no device on the LAN recognizes 74.125.24.108 as its IPv4 address, then a process called **next-hop resolution** occurs where the next, most logical jump in the sequence of leaps to the destination address is determined. ARP is again called upon to discover the next network's MAC address (identifier for the router, gateway, etc. which is establishing the LAN's presence), by way of the IPv4 address that the next network is already broadcasting publicly. This repeats, as many times as necessary, until the destination is finally reached – leaping across adjacent networks like a frog over lilypads.

Most devices now keep a stored table called the **ARP cache** which remembers recently learned IP-to-MAC address pairs – think of it almost like a phonebook or contact list. The ARP cache helps to facilitate the entire process, as it can reduce the amount of IP-to-MAC conversion requests that ARP must make if the IP-to-MAC address pairing is already known. This is most notably helpful during the next-hop resolution process, as it makes sense that many outgoing or incoming packets must make their hops through the adjacent, connected networks. These neighboring networks, as the ones being accessed the most, would obviously be familiar then. ARP also only works as a resolution protocol with IPv4 addresses – but IPv6 networked addresses have their own, respective, identically functional protocol: the **Neighbor Discover Protocol (NDP)**.

Every transmission the packets must make through a different LAN is commonly referred to as a "hop". In fact, if the reader would like to see this process occur in real-time, they can go to any Internet capable computer (Windows, macOS, or Linux) and open an application on the machine called the terminal, or Command Prompt on Windows. If you happen to be running a Linux-based operating system, ensure you have the `traceroute` command line utility installed using your distribution's accompanying package installer (`apt`, `dnf`, `pacman`, etc.); for Windows, the Command Prompt uses the built-in `tracert` function. And macOS happens to come pre-loaded with the Linux `traceroute` command. Once you are at the terminal / Command Prompt, enter Figure 7 for Linux / macOS and Figure 8 for Windows...

```
traceroute 74.125.24.108
```

Figure 7 – GNU/Linux command line utility to observe LAN "hops" from source to destination address

Note the amount of "hops" it takes for our tracer ping to reach its destination. Each of these is a different router running an inter-connected LAN.

```
>tracert 74.125.24.108
```

Figure 8 – the respective command to be entered for Windows systems

If you were to put in the command with a different IP address than our example one, it would probably take considerably less hops between different LANs.

That would be because our example Google SMTP / POP3 mail server IP address is (at the time of this report) located in Singapore. If you put in something like 8.8.8.8 as the IP address parameter for the trace route utility, it may take up to 10, maybe 15 hops to reach the destination (which is a Google DNS server in California). As opposed to our example mail server IP address which, if pinging / accessing from the United States, could be anywhere from 25 to 30, 35 hops. As you can probably gather, this is one of the critical factors in **latency**, or how long it takes for a system to respond after being pinged by a user. Often quantified in milliseconds (abbreviated as *ms*), this time interval is shown as part of the output for both trace route utility commands depicted and described above.

```
C:\Users\freax>tracert 8.8.8.8

Tracing route to dns.google [8.8.8.8]
over a maximum of 30 hops:

  1    <1 ms    <1 ms    <1 ms  192.168.0.1
  2    11 ms    22 ms    12 ms  10.60.214.74
  3    24 ms    13 ms    12 ms  po-103-rur01.peoria.il.chicago.comcast.net [68.87.232.217]
  4    19 ms    18 ms    19 ms  be-151-ar01.area4.il.chicago.comcast.net [162.151.93.49]
  5    19 ms    20 ms    19 ms  be-501-arsc1.area4.il.chicago.comcast.net [96.216.150.113]
  6    21 ms    18 ms    20 ms  c-50-171-117-134.unallocated.comcastbusiness.net [50.171.117.134]
  7    20 ms    21 ms    19 ms  142.250.209.141
  8    20 ms    32 ms    22 ms  216.239.51.145
  9    20 ms    20 ms    22 ms  dns.google [8.8.8.8]

Trace complete.
```

Figure 9 – Output of the Windows **tracert** command pinging IP address 8.8.8.8 in the Command Prompt, showing the "hops" between local area networks (LANs) to reach Google's CA DNS server

Once the email or data reaches the end of its journey, it need only validate and resolve the IP address of the destination, to the MAC address of the device / network that the data ends up at. A successful resolution and confirmation means success, and the last leg of the trip is underway. The information transmits itself through either a form of physical networking topology (LAN, Ethernet), or over a wireless connection of some sort (Wi-Fi, cellular networks), to arrive at its final destination in a digestible format; beyond 1's and 0's in the Transport Layer, beyond the slew of digits making up the variety of IP addresses, and beyond the high level abstraction of TCP headers and Ethernet frames. This is of course the Physical Layer (Layer 1) of the OSI model – the last layer where the data finally arrives at the endpoint in an un-abstracted, coherent, and digestible manner.

Conclusion

What the reader should be able to take away from this documentation is a foundational amount of knowledge in how data communications work, and how information travels through networks. They should now have a basic understanding of how protocols at each layer are utilized to encapsulate and handle the data as it moves, and how each protocol serves to better help the following layer's task(s), thereby facilitating the flow of data through abstraction along the way. This kind of information can be important as a basis for a number of jobs involving technology, including (but not limited to): systems administrator, network programmer, IT help desk / IT support, cybersecurity analyst, etc.

Beyond just the obvious benefit as an industry skill that a strong background in data communications and networking carries, it can be advantageous for a number of more interesting, personal uses. The opportunity to work as a freelance hacker of sorts exists in many online spaces, where one can be compensated for finding security and network flaws in a company or organization's IT infrastructure – a gig known as "bug bounty hunting".

At the intersection of networking and computing lies Linux, an open-source computer operating system noted specifically for its efficiency, customizability, and power within the domain of networking. This is the operating system that many servers run on, as that is one of its primary intended use cases. Linux is especially useful if undertaking a hobby known as "homelabbing", where someone sets up their own Linux machine running a private, accessible home-based server of their own. This private network can then be used as a virtual private network (VPN), a proxy server, a media / file sharing server, a gaming server, etc. This connection with networking that Linux has, based mainly on its history / roots in UNIX, has led to its reputation as the "hacker" operating system.

For more practical applications of this information, one could consider locking down their own home network by logging into their router, modem, or gateway and enabling certain features. Things like enabling MAC address filtering for devices to connect, ensuring all TCP / UDP ports are closed, tweaking the firewall settings – these are fairly simple security measures to enact on one's own network that would necessitate some amount of more advanced networking knowledge or practice.

The trouble is that data communications and computer networking really are skillsets that are one part knowledge and one part applied skills. This report gives the reader a taste and some surface level exposure to the deeper, finer, technical points of how the entire process occurs from an eagle's eye view. Gaining further, hands-on experience in networking requires application of this knowledge in a real-world context; whether professional or personal. That is when the connection between this text's information and the actual practice of computer networking will click, but if you've read thoroughly and attentively, you should be well prepared for the next steps in your cyber adventure!

References

Articles:

"Sage: Semi-Automatic Ground Environment Air Defense System." *MIT Lincoln Laboratory*, Massachusetts Institute of Technology, www.ll.mit.edu/about/history/sage-semi-automatic-ground-environment-air-defense-system. Accessed 22 Mar. 2026.

"From ARPANET to the Internet | Science Museum." *Science Museum*, 2 Nov. 2018, www.sciencemuseum.org.uk/objects-and-stories/arpanet-internet.

Books:

Forouzan, Behrouz A. *Data Communications and Networking*. McGraw-Hill, 2013.

Kozierok, Charles M. *The TCP / a Comprehensive, Illustrated Internet Protocols Reference.* Estados Unidos, No Starch, 2005.

Leiden, Candace, and Marshall Wilensky. *TCP/IP for Dummies*. Wiley Pub, 2009.

Graphics:

Figure 1: Website: SAGE: Semi-Automatic Ground Environment Air Defense System | Massachusetts Institute of Technology Lincoln Laboratory

Figure 2: Website: TCP/IP Network Model - Techprodezza

Figure 3: Website: MDE's DeviceNetworkEvents table [Part 2 — Connection* ActionTypes] | by Ethan Bowen | Medium

Figure 4: Website: TCP Header Packet Contents TCP Header Field Definitions (Left-to-Right... | Download Scientific Diagram

Figure 5: Website: UDP Header | Structure & Functions Explained for Networking

Figure 6: Website: Ethernet Protocol | IEE 802.3 Frame Format – learncisco.net